UniReg — University Course Registration System

Final Project Report

Alexandria University — Faculty of Engineering Computer and Systems Engineering Department Course: Requirements Engineering and System Modeling Submission Date: 6 May 2026

Team Members and Task Assignment

#	Name	Student ID	Main Responsibility
1	مصطفي اشرف سعد	23012069	Prisma schema, validator engine, enrollment + waitlist APIs, NextAuth setup, role middleware.
2	عمرو شريف سليمان	23011396	Student dashboard, course browse, schedule grid, transcript page, override + admin APIs.
3	مهيمن هاني محمد	23011572	Context, Use Case, and Activity diagrams. System Description section. Advisor override inbox UI.
4	محمد سلامة محمد علي الجبالي	23011134	State Machine, Gantt, and PERT diagrams. Requirements Engineering tables. Admin terms + courses pages.
5	البير عاطف شفيق	23011225	Admin sections page, audit log viewer, transcript PDF template. Final report assembly and PDF export.
6	احمد سالم السعيد	22010019	Login page, instructor roster page, database seed script. Video recording, editing, and submission packaging.

---

1. System Description

1.1 Project Title

UniReg — University Course Registration System.

1.2 Problem Description

Students at Alexandria University register for courses through a process that does not catch common problems on its own. Time clashes between sections, missing prerequisites, and full sections are handled by hand and create errors every term. Students need a system that checks all rules at the moment of registration and shows a clear answer.

1.3 System Objectives

1. Let students register and drop courses inside an open registration window.
2. Enforce prerequisites, time conflicts, credit-hour limits, and section capacity automatically.
3. Place students on a FIFO waitlist when a section is full and promote them when a seat opens.
4. Give advisors a simple way to approve or reject override requests.
5. Give admins control over terms, courses, sections, and registration windows.
6. Keep an audit record and a transcript that the student can download.

1.4 Stakeholders

Stakeholder	Role in the System
Student	Browses sections, registers, drops, joins waitlist, requests overrides, views transcript
Academic Advisor	Reviews override requests and approves or rejects them
Admin / Registrar	Manages terms, courses, sections, schedules, and views audit log
Instructor	Views the roster of students in their assigned sections
IT Support	Maintains the deployment and the environment

1.5 Work Breakdown Structure (WBS)

```
UniReg Project
├── 1. Project Initiation & Planning
│   ├── 1.1 Define project scope and objectives
│   ├── 1.2 Identify stakeholders
│   └── 1.3 Project scheduling and risk management
├── 2. Requirements Engineering
│   ├── 2.1 Stakeholder elicitation
│   ├── 2.2 Requirements specification (FRs & NFRs)
│   └── 2.3 Requirements validation
├── 3. System Design
│   ├── 3.1 Architectural design
│   ├── 3.2 Database schema design
│   ├── 3.3 UI/UX wireframing
│   └── 3.4 System Modeling (UML Diagrams)
├── 4. System Implementation
│   ├── 4.1 Database setup and migrations
│   ├── 4.2 Core Validator Logic & Backend APIs
│   ├── 4.3 Student Portal (Registration, Schedule)
│   ├── 4.4 Admin & Advisor Portals
│   └── 4.5 Waitlist engine & Audit logging
├── 5. Testing & Quality Assurance
│   ├── 5.1 Unit testing (Registration rules)
│   ├── 5.2 System integration testing
│   └── 5.3 User Acceptance Testing (UAT)
└── 6. Deployment & Handover
    ├── 6.1 Production deployment
    ├── 6.2 User manuals & documentation
    └── 6.3 Stakeholder training
```

---

2. Requirements Engineering

2.1 Requirements Engineering Process

We followed the standard four steps:

- **Elicitation.** We listed the people who use the system (student, advisor, admin, instructor) and wrote the main need each one has.
- **Specification.** We turned each need into a numbered requirement, split into Functional and Non-Functional, and labeled it as User or System.
- **Analysis.** We re-read the requirements to find sentences that were unclear or that pulled in two directions, and we resolved each one.
- **Validation.** We checked that every requirement is covered by a model and by working code in the demo.

2.2 Functional Requirements

ID	Requirement	Type
FR1	Students shall sign in with demo credentials and reach a role-specific dashboard.	User
FR2	Students shall browse available sections by department, level, day, and seat availability.	User
FR3	The system shall let a student enroll in a section when registration rules pass.	System
FR4	The system shall validate registration window, duplicate enrollment, prerequisites, time conflicts, and credit-hour cap before confirming enrollment.	System
FR5	The system shall place students on a FIFO waitlist when a section is full and auto-promote the next valid student when a seat opens.	System
FR6	Students shall view their schedule, current enrollments, and waitlisted sections.	User
FR7	Students shall drop enrolled sections before the drop deadline.	User
FR8	Students shall submit override requests and review their status history.	User
FR9	Advisors shall review pending override requests and approve or reject them.	User
FR10	Admins shall create and edit academic terms, including registration and drop windows.	User
FR11	Admins shall create and edit courses, prerequisites, and sections, including instructor assignment.	User
FR12	The system shall generate transcript PDFs and maintain an audit trail for enrollment and override actions.	System

2.3 Non-Functional Requirements

ID	Requirement	Type	Category
NFR1	Core demo actions should respond within 2 seconds under classroom demo load.	User	Performance
NFR2	Role-based authorization shall prevent cross-role access to protected routes.	System	Security
NFR3	Validation and audit behavior shall be deterministic for the same inputs.	System	Reliability
NFR4	The UI shall remain usable on laptop and mobile-width screens used in the demo.	User	Usability

NFR5	Shared rules shall be centralized in reusable modules to reduce maintenance overhead.	System	Maintainability
NFR6	Setup shall stay lightweight through SQLite, Prisma migrations, and seeded local accounts.	System	Portability

2.4 User vs System Requirements

User requirements describe what the user wants to do in plain language. System requirements describe what the software must do to make that happen. The Type column in the tables above marks each row.

- **User examples:** FR1, FR2, FR6, FR7, FR8, FR9, FR10, FR11, NFR1, NFR4.
- **System examples:** FR3, FR4, FR5, FR12, NFR2, NFR3, NFR5, NFR6.

2.5 Requirements Analysis

Ambiguities Found

ID	Original Ambiguity	Resolved Form	Resolution
A1	"Students can register for available courses."	"Available" means open window, no duplicate enrollment, passed prerequisites, no clash, within credit cap, and a seat or waitlist slot is open.	Converted into explicit validator checks.
A2	"Students may drop courses."	Drop is allowed only before `dropClosesAt`; refund logic is out of scope.	Added a clear policy boundary.
A3	"System should be fast."	Core actions target a 2-second response.	Converted into NFR1.

Conflicts Found

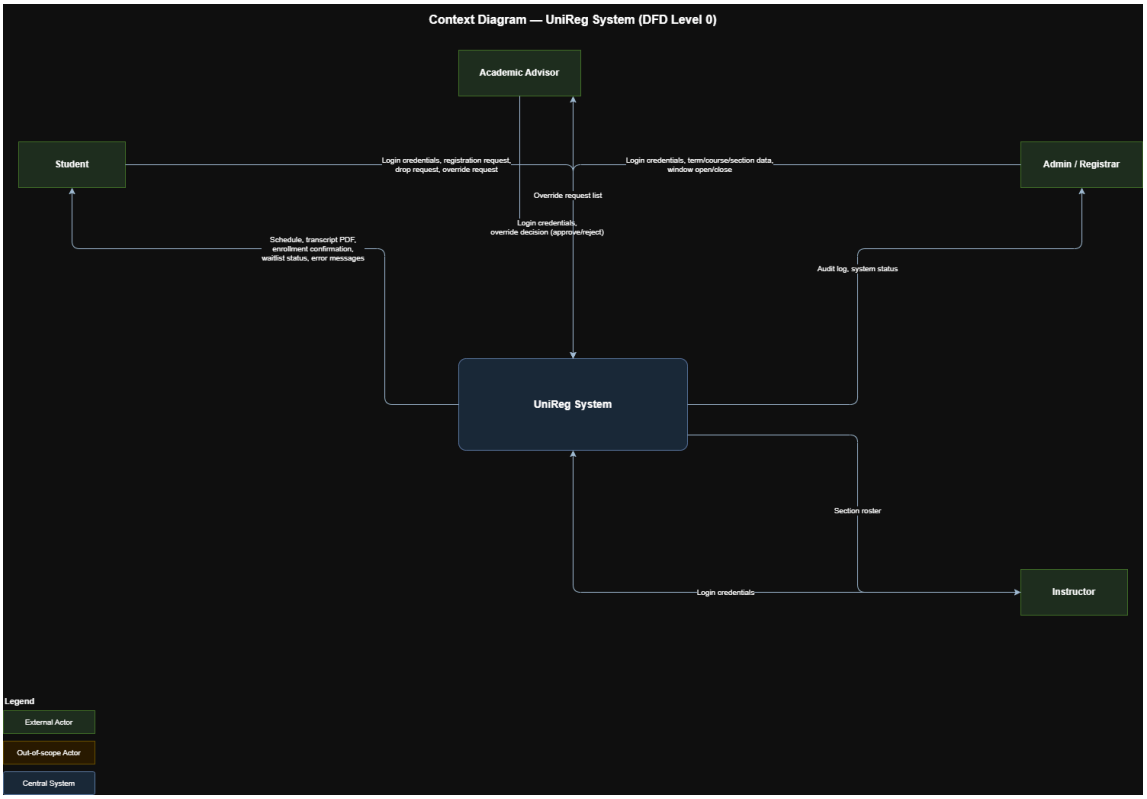
ID	Conflict	Resolution
C1	Strict prerequisite enforcement conflicts with advisor-approved exceptions.	Approved overrides bypass only the blocked rule and are logged.
C2	Capacity enforcement conflicts with fair registration after a drop.	Waitlist promotion happens only after a seat frees and revalidation passes.
C3	Broad enterprise integrations conflict with the demo time limit.	SSO, real email delivery, and registrar integrations are deferred.
C4	The instructor roster could push the official FR count past the rubric range.	It is implemented as a supplemental read-only feature and is not counted in FR1–FR12.

---

3. System Models

Each model below has one short paragraph for **why we use it** and one for **how it connects** to the others.

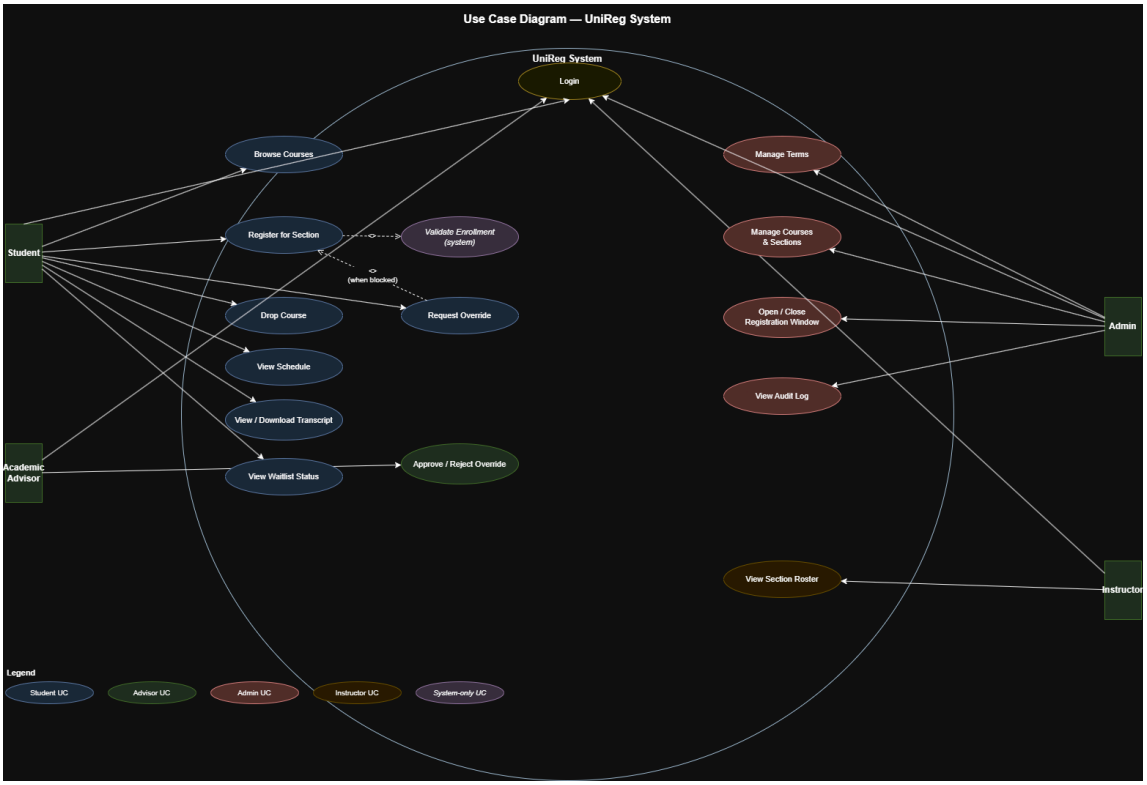
3.1 Context Diagram



Why used. Shows the system boundary in one picture. We see what is inside UniReg and what is outside, and which actors send or receive data.

How it connects. Names the four actors (Student, Advisor, Admin, Instructor) that the Use Case Diagram then breaks down into individual actions.

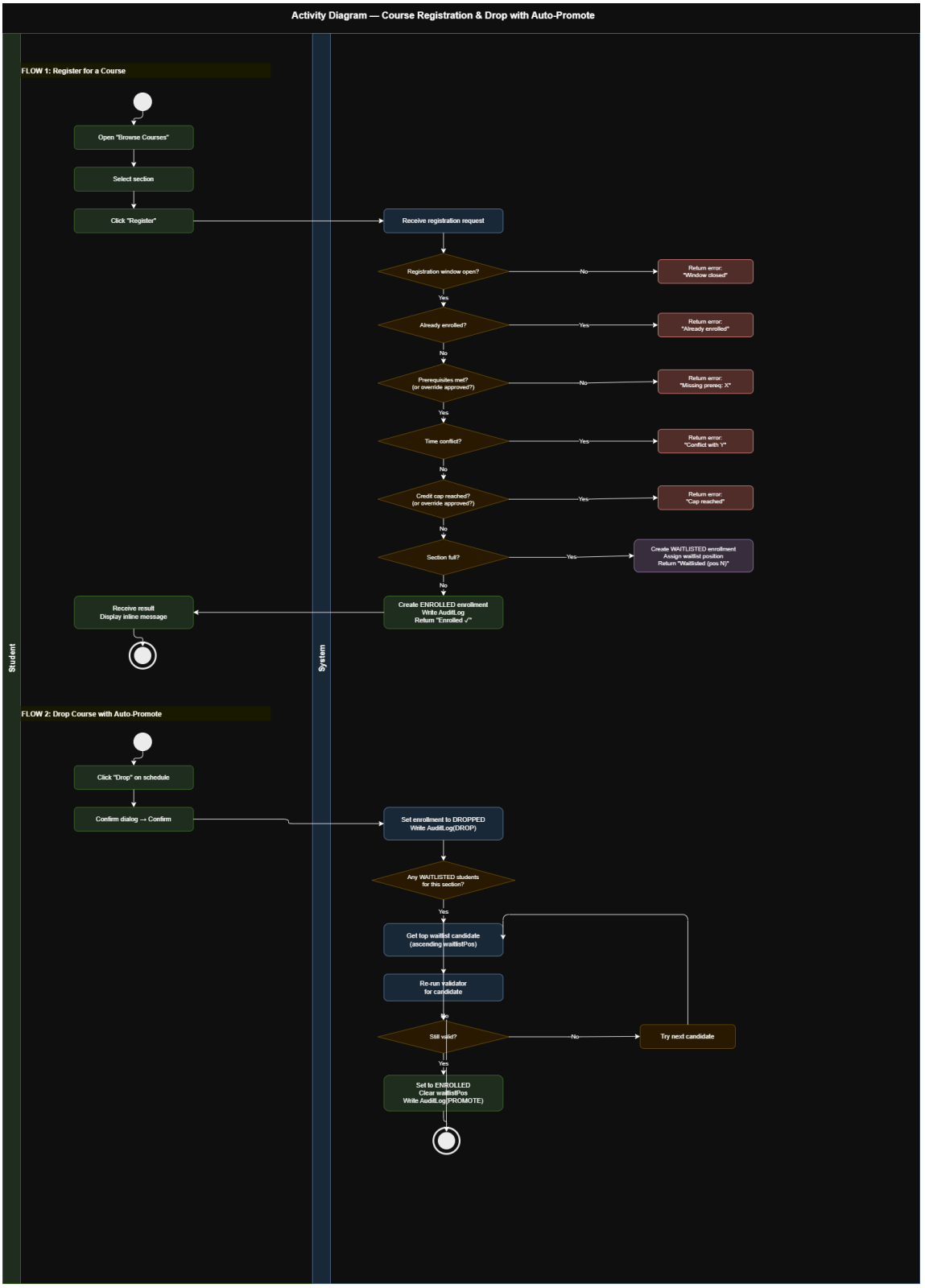
3.2 Use Case Diagram



Why used. Lists every action a user can take. `include` and `extend` show how actions depend on each other (for example, `Register for Section` includes `Validate Enrollment`, and `Request Override` extends it when the validation blocks the student).

How it connects. Each use case maps to one decision path in the Activity Diagram and to one transition in the State Machine.

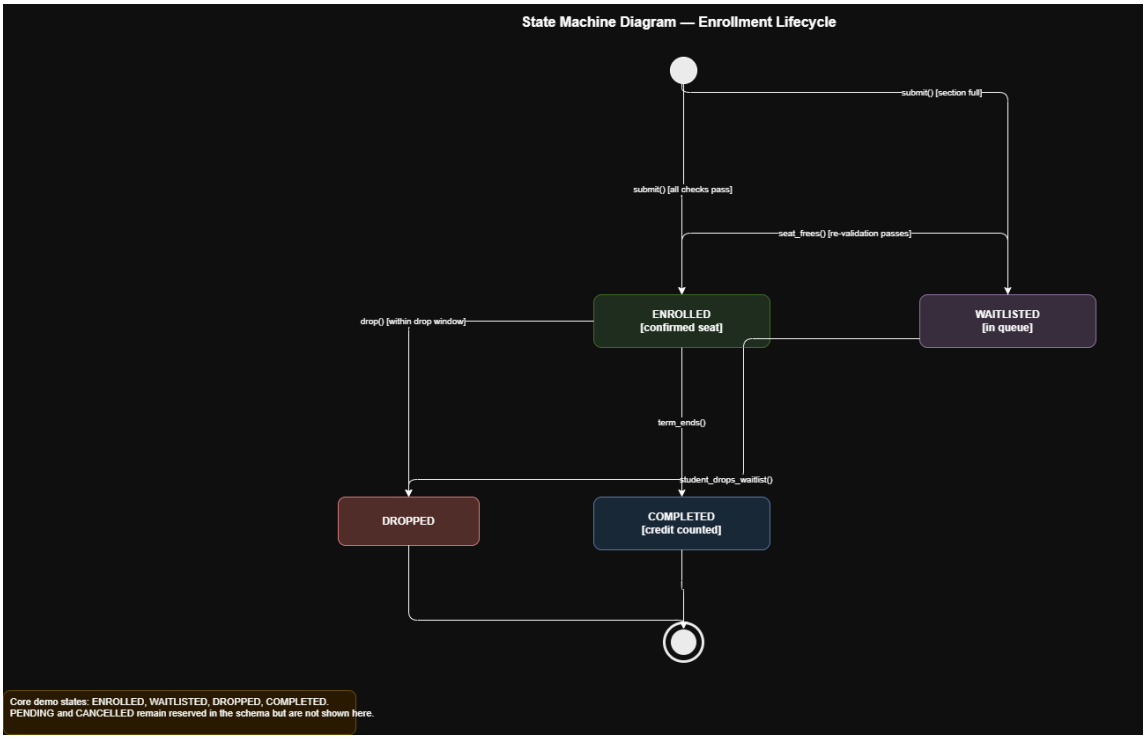
3.3 Activity Diagram



Why used. Shows the registration and drop flows step by step in two swimlanes (Student | System). Captures the order of the validator checks: window → duplicate → prerequisite → clash → credit cap → capacity.

How it connects. Each system step in this diagram writes one of the states shown in the State Machine and matches one of the API endpoints described in the demo section.

3.4 State Machine Diagram (Bonus)



Why used. Shows the lifecycle of one Enrollment record: ENROLLED, WAITLISTED, DROPPED, COMPLETED, and the transitions between them.

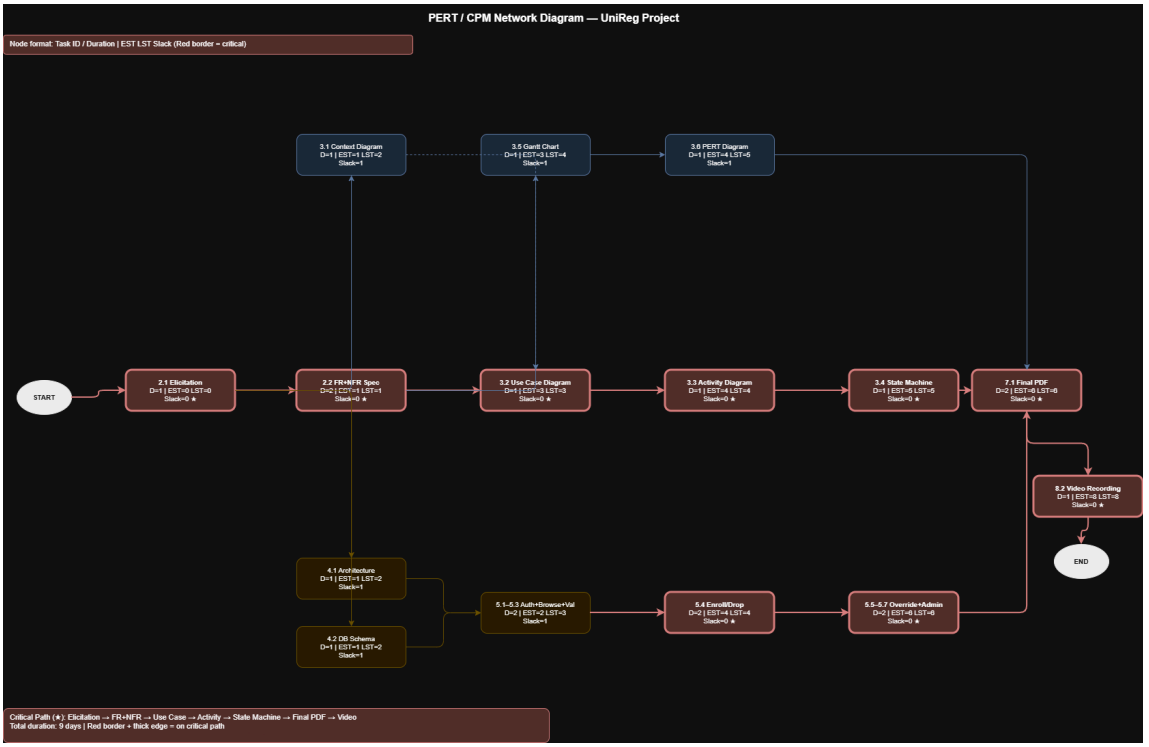
How it connects. Every transition is triggered by a use case from §3.2 and follows the path in the Activity Diagram. The states are stored as the `state` field in the database.

3.5 Gantt Chart

Why used. Shows the planned timeline for each WBS task and who owns it.

How it connects. The task names match the WBS in §1.5, and the dates support the critical path shown in the PERT Diagram.

3.6 PERT Diagram



Why used. Shows the order between tasks and marks the critical path that must finish on time for the project to ship.

How it connects. Uses the same tasks as the Gantt Chart and points to which deliverables (Implementation, Testing, Deployment) are on the critical path.

---

4. System Demo (Bonus)

4.1 Tech Stack

Layer	Choice
Framework	Next.js 16 (App Router)
Runtime	React 19
Language	TypeScript 5
Database	SQLite + Prisma 6
Authentication	NextAuth v5 (credentials provider)
Styling	Tailwind v4 + react-toastify
Validation	Zod 4
PDF Export	@react-pdf/renderer
Testing	Vitest 4

4.2 Architecture

The whole project is one Next.js repository. Pages, APIs, and authentication live in the same codebase. Role-based middleware blocks each route group from users who do not have the right role.

```
app/(auth)/login            sign-in page
app/(student)/student/*     dashboard, courses, schedule, transcript, overrides
app/(advisor)/advisor/*     override inbox
app/(admin)/admin/*         terms, courses, sections, audit
app/(instructor)/instructor roster view
app/api/*                   REST endpoints (JSON, session-cookie auth)
lib/validator.ts            shared enrollment rule engine
lib/transcript-pdf.tsx      PDF template
prisma/schema.prisma        database schema
```

4.3 Database Summary

The schema has six main tables: `User`, `Term`, `Course`, `Section`, `Enrollment`, `OverrideRequest`, plus an `AuditLog` table. Enrollments hold the state (`ENROLLED`, `WAITLISTED`, `DROPPED`, `COMPLETED`). Sections store their schedule as a small JSON array of day + start/end + room.

4.4 Feature Walkthrough (FR1–FR12)

FR	Where it appears in the demo
FR1	Login page → dashboard for the matching role
FR2	Student → Browse Courses (filters by department, level, day, seats)
FR3	Browse Courses → Register button on a section card
FR4	Validator runs server-side before any enrollment is written
FR5	Full sections show "Join Waitlist"; on a drop the next valid student is promoted in the same transaction
FR6	Student → My Schedule (weekly grid + waitlist list)
FR7	My Schedule → Drop button (only before `dropClosesAt`)
FR8	Student → Overrides (submit + history)
FR9	Advisor → Override Inbox (approve / reject)
FR10	Admin → Terms (create, edit, set active, set windows)
FR11	Admin → Courses and Sections (CRUD + prerequisites + instructor)
FR12	Student → Transcript page + PDF download; every state change writes an `AuditLog` row

Supplemental: Instructor → Roster shows the list of students enrolled in each assigned section. Read-only and not counted in FR1–FR12.

4.5 Demo Scope Limits

The following are out of scope on purpose: real email delivery, university SSO, grade entry, fees and refunds, priority-based waitlist rules.

4.6 How to Run

```
npm install
npx prisma migrate dev --name init
npx prisma db seed
npm run dev          # http://localhost:3000
```

Demo accounts (password `demo1234` for all):

Email	Role
student@alex.edu	Student
advisor@alex.edu	Advisor
admin@alex.edu	Admin
instructor@alex.edu	Instructor

---

5. Conclusion

UniReg covers every item asked for in the brief: full system description, requirements engineering process, twelve functional and six non-functional requirements split between user and system, ambiguity and conflict analysis, and the six required models. The two bonus items — the State Machine Diagram and a working coded demo — are both delivered.

The main design tradeoff was scope: we kept the system focused on the registration loop instead of adding fees, grades, or external integrations, so the rules engine and the audit trail could be done well. With more time we would add real email notifications, a calendar export for the schedule, and a mobile-friendly version of the admin pages.

All deliverables (this report, the diagrams sheet, the code, and the video) reflect the same working state of the project.